

软件详细设计说明书：TravelMate（伴游）智能旅游平台

软件详细设计说明书：TravelMate（伴游）智能旅游平台

1 引言

- 1.1 编写目的
- 1.2 背景
- 1.3 术语表
- 1.4 参考文档

2 系统结构设计及子系统划分

- 2.1 典型业务活动图（以智能行程规划为例）
- 2.2 典型业务顺序图（以订单支付为例）

3 功能模块设计

- 3.1 AI 行程生成模块
- 3.2 订单库存管控模块
- 3.3 社区内容推荐模块
- 3.4 基础订单管理模块
- 3.5 用户认证与安全模块
- 3.6 目的地住宿与本地生活模块
- 3.7 旅途社区与用户中心模块
- 3.8 管理后台与可观测性模块

4 界面设计

- 4.1 外部界面设计
 - 4.1.1 第三方与外部服务接口
 - 4.1.2 统一响应格式
 - 4.1.3 核心 API 端点清单
 - 4.1.4 接口限流设计
- 4.2 内部界面设计
 - 4.2.1 服务间通信规范
 - 4.2.2 数据库设计
- 4.3 用户界面设计

5 异常处理设计

- 5.1 出错信息管理
- 5.2 故障预防与补救
- 5.3 系统维护设计

6 性能优化和安全设计

- 6.1 性能优化
- 6.2 安全与风险控制设计

7 项目测试计划

1 引言

1.1 编写目的

本文档旨在对“TravelMate（伴游）”智能旅游平台进行详细设计说明。在概要设计的基础上，进一步明确系统各模块的实现算法、数据结构、接口细节及业务逻辑。

本文档的预期读者包括：前端开发工程师、后端开发工程师、测试工程师、运维工程师以及项目管理人员。通过本说明书，可指导系统编码实现、模块联调以及后续测试与维护工作。

1.2 背景

- **软件名称：**伴游（TravelMate）出行旅游平台
- **任务提出者：**软件工程基础 2026 春课程组
- **开发者：**Miracle 开发小组
- **预期用户：**旅游游客、旅游内容创作者、平台运营与管理人员。
- **运行与部署环境：**
 - **开发与测试环境：**团队成员个人计算机，支持 Chrome、Edge 等主流 Web 浏览器。
- **核心技术栈选型：**
 - **前端：**Vue 3 + Vite + Element Plus + Pinia + Axios。
 - **后端：**Java 21 + Spring Boot 3.5.13 + MyBatis-Plus 3.5.7。
 - **认证：**Spring Security + jjwt 0.11.5（HS256 对称密钥，24h 过期）。
 - **数据存储与中间件：**MySQL 8.0、Redis 缓存中间件。
 - **AI 能力接入：**支持工具调用（Skills）和 JSON Output 的 DeepSeek API（OpenAI 兼容协议，默认模型 `deepseek-v4-flash`）。

1.3 术语表

术语	全称	定义/说明
OTA	Online Travel Agency	在线旅游代理平台
AI Agent	Artificial Intelligence Agent	具备环境感知、推理与任务执行能力的智能代理系统
Agent	—	具备任务理解、工具调用和多轮对话能力的智能代理
JWT	JSON Web Token	用于前后端身份认证与会话保持的令牌机制
RBAC	Role-Based Access Control	基于角色的权限访问控制模型
UGC	User Generated Content	用户生成内容，如游记、评论、图片等
QPS	Queries Per Second	每秒请求数，用于衡量系统吞吐能力
RAG	Retrieval-Augmented Generation	检索增强生成技术，用于提升大模型回答准确性
Mock	—	模拟数据或模拟接口，用于课程项目中替代真实商业接口
RESTful API	—	基于 HTTP 协议的资源风格接口设计方式
Skills	—	大模型可调用的外部工具或内部函数能力
Redis	—	键值缓存中间件，用于缓存、限流、延时控制和热点状态存储
MyBatis-Plus	—	基于 MyBatis 的 ORM 增强框架

1.4 参考文档

- 《软件工程基础 2026 春大作业要求》
- 《5 组—软件需求规格说明》
- 《软件概要设计说明书》
- 《软件工程导论》
- DeepSeek API 官方技术文档
- Spring Boot、Vue 3、Vite、MySQL、Redis、Element Plus、ECharts 等官方文档
- 携程、同程、去哪儿、小红书等产品的公开功能调研结果

2 系统结构设计及子系统划分

本项目采用前后端分离的系统架构。后端基于 Java 21 + Spring Boot 3.5.13 + MyBatis-Plus 3.5.7 构建 RESTful 风格的服务端接口，前端采用 Vue 3 + Vite + Element Plus 实现响应式交互界面。在数据交互上，各模块优先统一核心数据模型（用户、订单、资源、消息等），确保不同成员开发的模块能够顺利进行集成联调。



按照项目团队的开发分工与核心业务流，我们将系统整体划分为以下五个核心子系统：

1. 大交通票务与订单子系统（模块负责人：A 邹林利）

- 功能职责**：负责平台核心大交通资源的预订与基础订单生命周期管理。
- 核心设计**：不接入真实商业票务系统，机票与火车票数据优先采用本地 Mock 数据与数据库初始化数据支撑。实现车次/航班实时查询、多平台模拟比价、历史价格走势展示。统一管理平台基础订单模型，包含订单状态流转控制、模拟支付接口对接、库存预占调用以及退款处理逻辑。

2. 目的地住宿与本地生活子系统（模块负责人：B 莫谨瑞）

- **功能职责：**以最小可交付范围负责酒店、景点基础浏览，以及行后评价提交与展示。
- **核心设计：**实现酒店多维度搜索过滤、酒店 / 景点详情展示、基础五星图文评价提交与列表展示。房务预订中的库存防超卖逻辑由 A 的订单库存管控模块统一设计；景点门票购买入口已在目的地流程中提供，核销二维码作为订单/凭证展示能力处理；商家回复、评价举报处理、酒店 / 景点后台维护由 E 的管理后台负责。

3. AI 智能规划与 Agent 服务子系统（模块负责人：C 陈一鸿）

- **功能职责：**负责平台所有与大语言模型相关的智能化场景以及全局消息触达。
- **核心设计：**统一封装并异步调用 DeepSeek API。行程规划模块强制使用 **JSON Output** 格式化返回结果，保障前端渲染的结构化；AI 客服 Agent 结合自然语言意图识别与 **Skills/内部工具调用**（如查询平台站内酒店数据），实现多轮智能问答。此外，负责统筹平台实时通知服务（站内信、红点未读逻辑），并在行程与门票场景中展示景点预订入口和订单/凭证信息。

4. 旅途社区与用户中心子系统（模块负责人：D 杜新诚）

- **功能职责：**承担全平台的用户认证鉴权、社交互动与内容沉淀。
- **核心设计：**实现基于 **JWT** 的安全登录注册与用户信息管理；提供 UGC 内容发布工具（多图上传、地理位置打卡）；搭建兼容图文与视频的双列瀑布流交互界面；构建包含关注/粉丝体系、点赞收藏与树状评论结构的社交关系链。

5. 管理后台与可观测性子系统（模块负责人：E 李科）

- **功能职责：**面向平台管理员，提供资源维护、安全审核与系统运行状态监控。
- **核心设计：**搭建基于 **RBAC（基于角色的访问控制）** 的权限与安全架构；实现轻量数据可观测仪表盘（如实时订单大屏、本地访问量 QPS 监控、报错日志看板）；构建内容与安全审核防线（社区图文敏感词自动过滤 + 人工审核工作流）；提供大交通、酒店、景点基础数据的后台 CRUD 管理能力，以及商家回复、评价举报工单处理等运营侧功能。

子系统间的协同关系说明：

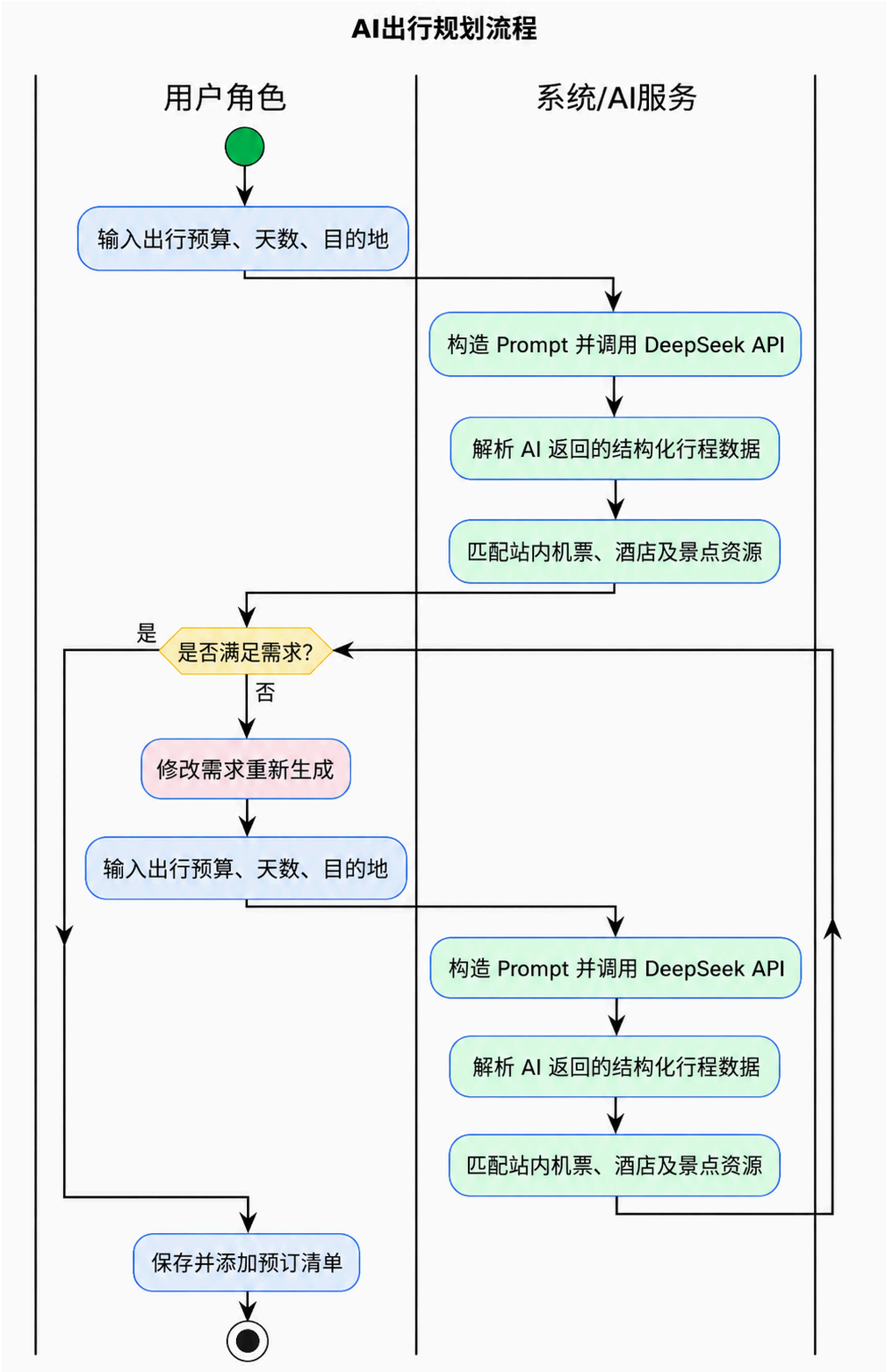
- 子系统 A 与 B 重点围绕**交易履约流程和库存控制模型**进行底层协同；
- 子系统 C 在生成行程与智能问答时，需跨模块调用 A/B 提供的资源检索接口（Mock 数据共享）；
- 子系统 D 生成的 UGC 内容数据，需实时推送到子系统 E 的审核流中完成安全合规校验。

TravelMate 五大子系统协同关系图

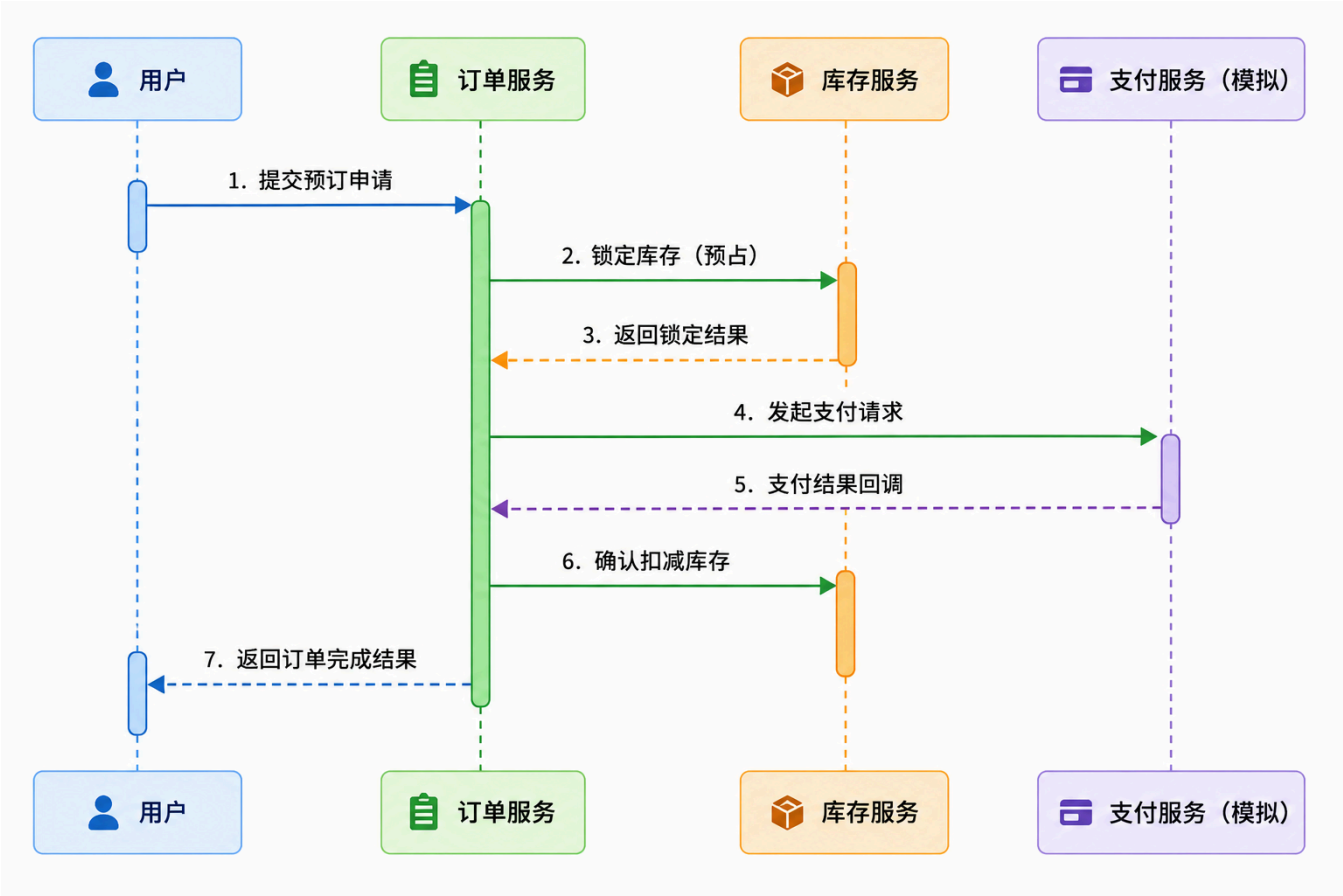
五大子系统协同工作，构建完整的智能旅行服务平台



2.1 典型业务活动图（以智能行程规划为例）



2.2 典型业务顺序图（以订单支付为例）



3 功能模块设计

3.1 AI 行程生成模块

- 模块描述**: 根据用户输入生成个性化旅游行程，并与平台 Mock 资源进行关联匹配。
- 模块编号**: TM-MOD-01
- 输入**: 目的地 (String)、天数 (Int)、预算范围 (Float)、出行人数 (Int)、偏好标签 (List)
- 处理**:
 - 对用户输入进行合法性校验。
 - 构造标准化 Prompt (模板化 + 变量参数填充)。
 - 异步调用 DeepSeek API 获取行程推荐结果。
 - 使用 JSON Output 约束，对返回结果进行严格的结构化校验。
 - 对不完整字段进行二次补全，或抛出异常提示用户重新生成。
 - 调用内部服务接口 (Mock 数据查询)，匹配对应的酒店、景点及交通资源。
 - 规范化记录**: 对调用过程中的 Prompt、多轮 Session 状态以及调用的业务数据 Skills，进行统一日志打印与留存。
- 算法与调用描述**:

采用 Prompt Engineering + RAG (检索增强生成) 策略，结合本地 Mock 景点数据与历史热门行程。要求模型按 `{"type": "json_object"}` 格式输出，以便前端解析渲染。

开发过程中所有与大模型相关的用途、Prompt 模板、Session 上下文管理逻辑以及 Skills 实现机制，必须由模

块负责人形成单独的 Markdown 附件进行记录提交，确保模型使用的可解释性与可追溯性。

- **输出：**行程 JSON 数据（按天划分）、推荐资源列表、行程摘要信息。



3.2 订单库存管控模块

- **模块描述：**根据小并发稳定性设计的目标，负责机票、酒店等资源在预订过程中的防超卖控制与数据一致性保证。

- **模块编号：**TM-MOD-02

- **输入：**商品 ID（Mock 资源主键）、预订数量、用户 ID

- **处理：**

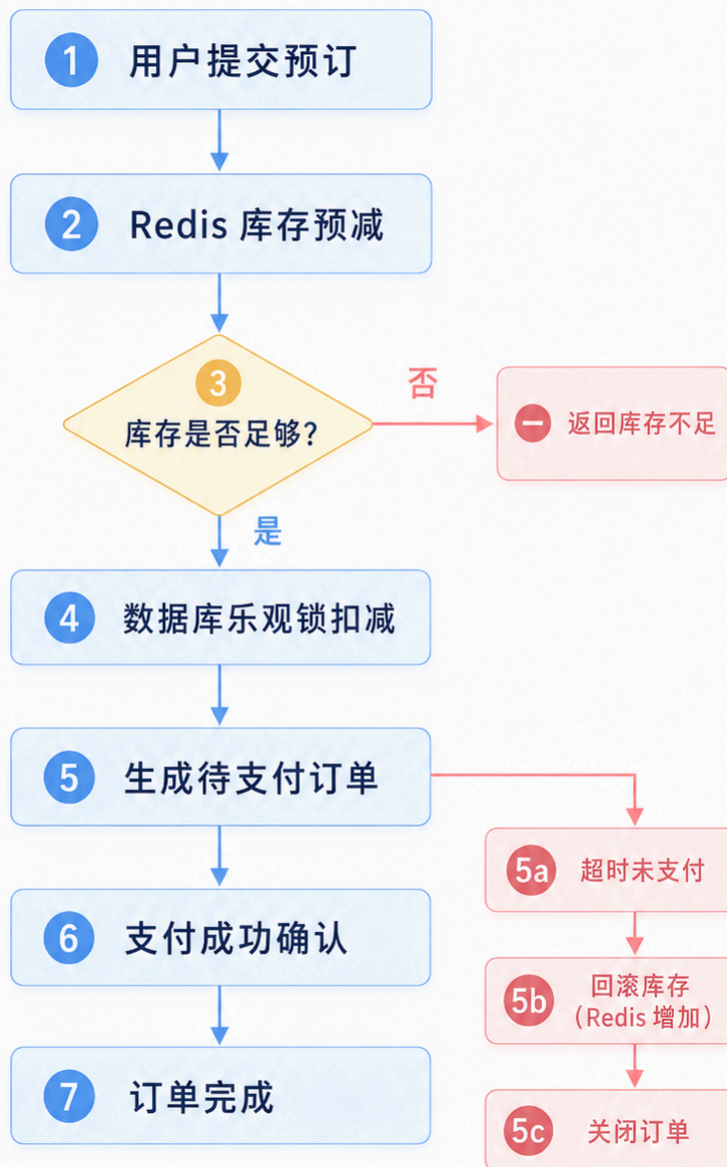
1. **Redis 缓存预减：**接收请求后，优先在 Redis 中进行库存扣减操作，快速拦截非法/超量请求，提升系统在瞬时小并发下的响应速度。
2. **数据库原子校验：**订单落库时，放弃重量级的行级悲观锁，采用数据库原子更新语句进行库存校验（例如：`UPDATE tm_hotel_room SET available_rooms = available_rooms - 1 WHERE id = ? AND available_rooms > 0`）。
3. 生成待支付订单，标记库存处于“预占”状态。
4. 接收到支付成功回调后，正式确认为已售库存。
5. 超时未支付的订单（如 15 分钟），通过延时任务触发库存回滚，恢复 Redis 与数据库中的剩余可用量。

- **算法策略描述：**

不使用企业级庞大复杂的分布式锁方案，采用“Redis 缓存预减 + 数据库原子更新语句 + 超时回滚”的组合策略。该策略既能有效防止资源超卖现象，又能充分验证小并发场景下系统处理关键业务的并发安全性。

- **输出：**锁定成功 / 失败状态标识、最新库存余额。

库存防超卖流程图



Redis 预减 + 数据库乐观锁 + 超时回滚

3.3 社区内容推荐模块

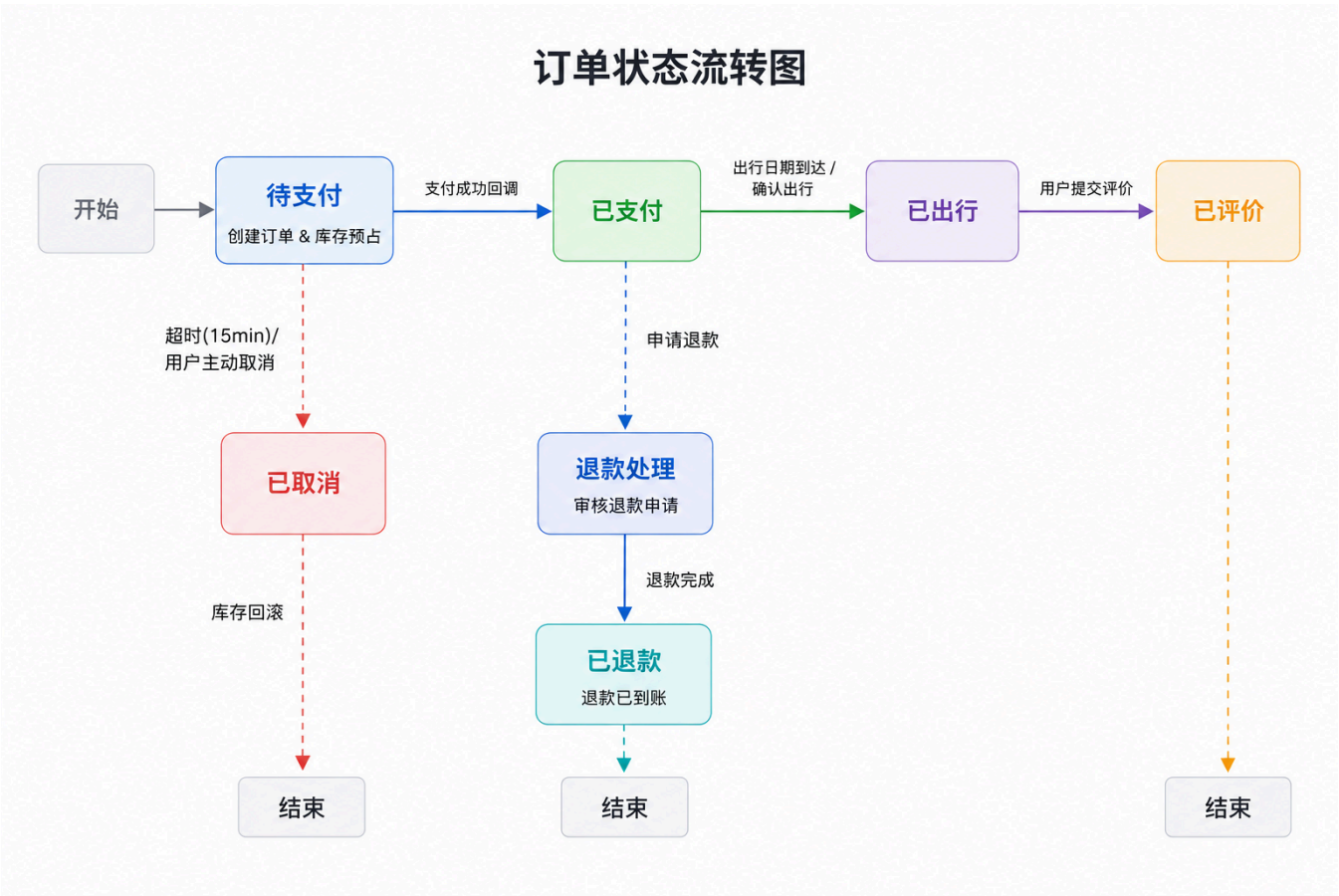
- **模块描述：**为用户提供基于 UGC 内容的个性化推荐流。
- **模块编号：**TM-MOD-03
- **输入：**用户行为数据（浏览、点赞、收藏）、内容标签信息。
- **处理：**
 1. 构建基于本地业务库的简易用户兴趣标签模型。
 2. 基于标签库进行协同过滤基础推荐。
 3. 按热度因子（点赞量/评论量）与时间因子进行加权衰减排序。
 4. 过滤命中后台违规词库的低质量或敏感内容。

- **算法策略描述：**采用“标签匹配 + 热度排序 + 时间衰减”的轻量级混合推荐策略，在不引入复杂大数据组件的前提下实现基础个性化推荐，支撑双列瀑布流展示。
- **输出：**推荐内容 ID 列表。

3.4 基础订单管理模块

- **模块描述：**统管大交通与酒店业务线的生命周期，构建业务闭环的基石。
 - **模块编号：**TM-MOD-04
 - **输入：**各业务线生成的预留单、外部支付指令（模拟）、用户侧操作指令（取消/确认出行/评价）。
 - **处理：**
 1. 统一订单模型落库：生成全局唯一的订单流水号，聚合资源信息与用户信息。
 2. 模拟支付接口（Mock）处理：不接入微信/支付宝等真实商业支付系统，而是由系统内部提供统一的“支付确认”Mock 接口（或本地桩代码），模拟支付成功并主动触发 Webhook 回调服务。
 3. 核心订单状态机流转控制：管理订单的阶段演进，校验状态跃迁的合法性。定义基础流转路径：【待支付】->【已支付 / 已取消】->【已出行（履约）】->【已评价 / 退款处理】。
 4. 模拟生成电子票据及行程单导出功能。
 - **算法与业务逻辑描述：**

基于状态机（State Machine）模式实现订单流转。每次状态变更须校验 `expected_current_state`（预期当前状态），防止因小并发点击导致的状态乱序。通过对支付环节进行 Mock 模拟，将重点集中于验证从“行前规划、下单预占”到“行中履约”、“行后评价”的完整业务闭环真实性。
 - **输出：**统一订单详情 DTO、状态变更操作记录。
- 订单状态机流转如下：



3.5 用户认证与安全模块

- **模块描述：**负责全平台统一身份认证、会话管理与角色权限控制，是所有业务模块的安全基础。
- **模块编号：**TM-MOD-05
- **输入：**注册请求（用户名 / 邮箱 / 手机号 / 密码）、登录请求（用户名 + 密码）、HTTP 请求 Header 中的 Bearer Token。
- **处理：**
 1. **注册流程：**接收注册信息，进行格式校验（长度、邮箱格式等），检查用户名唯一性，使用 BCrypt 对密码进行加盐哈希加密后落库（`tm_user` 表）。
 2. **登录流程：**查询用户，用 `BCryptPasswordEncoder.matches()` 校验密码，校验通过后使用 jjwt 0.11.5（HS256 对称密钥）生成 JWT，过期时间 24h，连同用户基础信息一并返回给前端。
 3. **JWT 拦截验证：**`JwtFilter` 在 Spring Security 过滤链中拦截每个请求，从 `Authorization: Bearer <token>` Header 提取 token，解析有效性与过期时间，将 username 设入 `SecurityContextHolder`。
 4. **权限控制（RBAC）：**`SecurityConfig` 按接口路径配置白名单（`/user/register`、`/user/login`、航班 / 酒店只读 GET 等），其余接口均需认证。管理员接口额外校验 `role == 1`。
 5. **UserContext 工具：**Controller/Service 层通过 `UserContext.getCurrentUserId()` 获取当前登录用户 ID，内部从 `SecurityContextHolder` 取 username 再查库。
- **关键设计决策：**
 - JWT 使用每次重启重新生成的内存对称密钥，旧 token 重启后全部失效（课程项目场景可接受，保证实现简单）；
 - 密码不可逆 BCrypt 存储，满足 OWASP 密码安全要求；
 - `User` 实体 `@TableName` 值为 `"tm_user"`，非 MyBatis-Plus 默认推导的 `"user"`。
- **输出：**JWT Token（登录成功）、用户详情 DTO、403/401 JSON 错误响应。

JWT 登录认证流程



3.6 目的地住宿与本地生活模块

- **模块描述：**提供酒店 / 景点浏览与基础评价功能；酒店订单库存一致性由 A 的订单模块协同保障，举报处理等运营功能由 E 的管理后台维护。
 - **模块编号：**TM-MOD-06
 - **输入：**城市 / 关键词、入住 / 退房日期、房客数量（酒店搜索）；景区 ID（景点详情）；评分 + 图文内容（评价提交）。
 - **处理：**
 1. **酒店搜索：**多条件组合查询（城市、日期、价格区间、星级、设施标签），返回酒店列表及首图摘要。
 2. **房型展示与库存展示：**查询指定酒店的所有房型，实时展示剩余可用库存（从 Redis 缓存或数据库查询）。
 3. **房务预订 & 防超卖：**接口入口保留在酒店订单流程中，具体 Redis 预减、数据库原子扣减、订单状态流转由 TM-MOD-02 / A 模块统一实现，B 只提供酒店与房型基础数据支持。
 4. **景点 / 本地玩乐：**提供景点列表与详情展示；门票购买入口通过 `/api/attraction/{id}/ticket` 提供，订单/凭证展示用于演示核销流程。
 5. **评价体系：**B 负责用户 1-5 星评分 + 图文评价提交与列表展示；商家回复（`tm_reply`）、评价举报（`tm_review_report`）及举报工单处理由 E 模块统一维护。
 - **关键实体：**`tm_hotel`、`tm_hotel_room`、`tm_hotel_order`、`tm_attraction`、`tm_review`、`tm_reply`、`tm_review_report`。
 - **输出：**酒店 / 景点列表 JSON、酒店 / 景点详情 JSON、基础评价提交与列表结果。
-

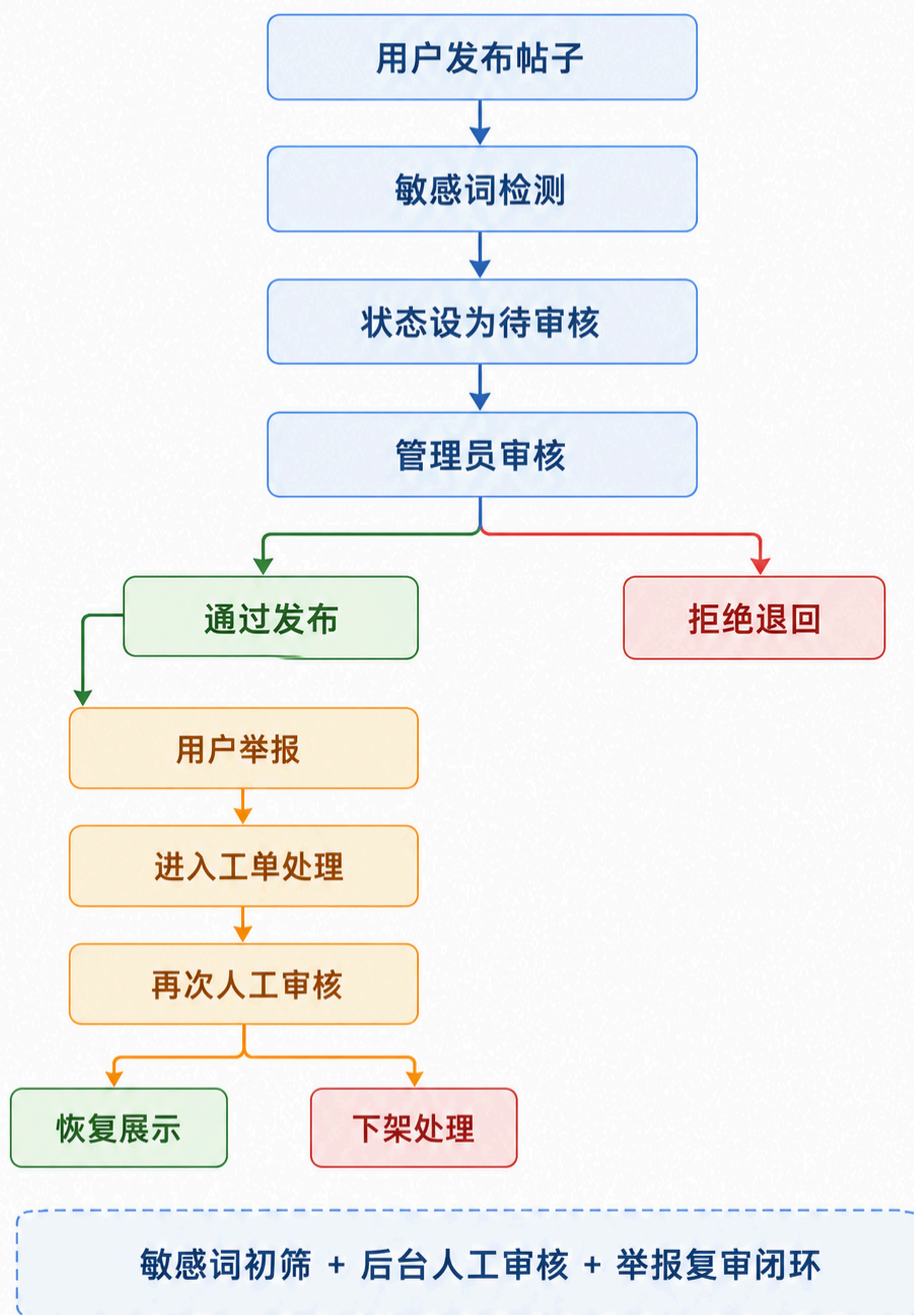
3.7 旅途社区与用户中心模块

- **模块描述：**承担全平台 UGC 内容发布、社交关系与互动，以及用户个人中心管理。
 - **模块编号：**TM-MOD-07
 - **输入：**图文帖子（标题 + 正文 + 图片 URL + 话题标签）、点赞 / 收藏 / 评论请求、关注 / 取关请求。
 - **处理：**
 1. **内容发布：**接收帖子创建请求，进行敏感词过滤（`SensitivewordService`），过滤通过后写入 `tm_post`，状态置为“审核中”（0）或“已发布”（1）；图片文件上传至本地存储，URL 写入帖子字段。
 2. **内容流：**首页推荐流按热度（点赞 + 评论加权）+ 时间衰减排序；关注流按 `follow` 关系过滤用户发布的帖子，分页返回。
 3. **互动行为：**点赞、收藏写入 `tm_like_record` / `tm_favorite_record`，同时更新 `tm_post` 中的计数缓存；评论采用父子树状结构（`parent_id`），写入 `tm_comment`。
 4. **关注关系：**`tm_follow` 记录 `follower_id` → `following_id`，关注成功后推送站内通知。
 5. **用户中心：**支持头像上传（文件接口）、个人资料修改、查看我的发布 / 点赞 / 收藏列表，以及修改密码（触发 Token 吊销）。
 - **关键实体：**`tm_post`、`tm_comment`、`tm_like_record`、`tm_favorite_record`、`tm_follow`、`tm_user`。
 - **输出：**帖子列表 / 详情 JSON、互动操作结果、用户主页数据 DTO。
-

3.8 管理后台与可观测性模块

- **模块描述：**面向管理员的运营管理、内容审核、系统监控与权限控制。
- **模块编号：**TM-MOD-08
- **输入：**管理员操作指令（资源 CRUD、内容审核、用户管理）、定时采集任务触发信号。
- **处理：**
 1. **资源管理：**航班、火车、酒店、房型、景点的后台增删改查，支持批量导入与库存调整。
 2. **内容审核：**对 `status=0`（待审核）的帖子进行人工审核，通过则置 `status=1`，拒绝则置 `status=2` 并记录原因；支持一键封禁用户。
 3. **优惠券管理：**创建满减 / 折扣优惠券（`tm_coupon`），设置发行量、使用规则、有效期；查看用户领券记录（`tm_user_coupon`）。
 4. **系统日志与 AOP 审计：**`SysLogAspect` 通过 AOP 自动拦截所有 `com.travelmate.controller` 下的方法，记录方法名、参数摘要、耗时、操作用户、IP 及成功 / 失败状态到 `sys_log` 表，日志落库失败不影响业务。
 5. **接口限流（RateLimiter）：**`@RateLimiter(maxRequests, timewindowSeconds)` 注解作用于高频写接口（下单、支付），`RateLimiterInterceptor` 基于 Redis 计数器 + IP 拦截超限请求，返回 HTTP 429。
 6. **可观测大屏：**统计在线用户数、订单总量、今日 GMV、接口平均响应时长、错误日志列表、热门目的地 TOP10；数据来源为 `sys_log` 聚合 + Redis 实时计数，前端使用 ECharts 可视化展示。
 7. **举报工单：**查看 `tm_review_report` 举报列表，支持驳回或删除被举报评价。
 8. **敏感词维护：**动态添加 / 删除敏感词库，立即生效于发帖审核流程。
- **关键实体：**`sys_log`、`tm_coupon`、`tm_user_coupon`、`sys_sensitive_word`、`tm_review_report`。
- **输出：**管理操作结果、审核结果、可观测统计 JSON（供前端 ECharts 渲染）。

社区内容审核流程图



4 界面设计

4.1 外部界面设计

4.1.1 第三方与外部服务接口

- **支付接口**：不接入真实商业微信/支付宝支付，而是采用内部模拟支付逻辑或本地桩代码（Mock）实现 Webhook 回调机制，专注于验证支付完成后的订单状态机流转与库存真实扣减。
- **票务与酒店资源接口**：大交通与酒店数据优先采用 Mock 接口响应和数据库初始化测试数据，以此支撑前端页面的搜索与预订交互。

- **AI 接口**：统一封装 DeepSeek API 调用接口，采用 OpenAI 兼容协议；认证方式为 Bearer Auth。对于行程生成等结构化场景，强制使用 **JSON Output**；对于内部业务联动场景，使用 **Skills 工具调用**（如将"查询本地 Mock 航班数据库"注册为工具供大模型调用），确保 AI 结果可控、可解释。
- **地图接口**：轻量调用第三方地图 API（如高德/百度），仅用于展示行程路线节点与打卡位置，而不实现复杂的实时轨迹计算。
- **文件接口**：支持社区图文图片的上传与获取，在前端或网关层进行图片压缩处理，降低服务器存储与带宽压力。

4.1.2 统一响应格式

前后端统一使用 `Result<T>` 封装所有响应：

```
{
  "code": 200,
  "msg": "success",
  "data": {}
}
```

前端 Axios 拦截器：`code === 200` 时直接返回 `data` 字段；非 200 抛出 Error；401 / 403 自动清除 Token 并跳转 `/login`。

4.1.3 核心 API 端点清单

用户认证接口

方法	路径	描述	认证
POST	<code>/user/register</code>	注册	公开
POST	<code>/user/login</code>	登录，返回 JWT	公开
GET	<code>/user/me</code>	获取当前用户信息	需登录
POST	<code>/user/password</code>	修改密码	需登录
GET	<code>/api/user/profile/{username}</code>	查看用户主页	公开
PUT	<code>/api/user/profile/update</code>	修改个人资料	需登录

大交通接口

方法	路径	描述	认证
GET	<code>/api/flight/search</code>	搜索航班（出发地、目的地、日期）	公开
GET	<code>/api/flight/{id}</code>	航班详情	公开
GET	<code>/api/train/search</code>	搜索车次	公开
GET	<code>/api/train/{id}</code>	车次详情	公开
GET	<code>/api/train/transfer</code>	中转方案推荐	公开

方法	路径	描述	认证
POST	/api/order/flight/create	创建机票订单（含 @RateLimiter）	需登录
POST	/api/order/train/create	创建火车票订单（含 @RateLimiter）	需登录
POST	/api/order/{orderNo}/pay	模拟支付	需登录
POST	/api/order/{orderNo}/cancel	取消订单	需登录
GET	/api/order/list	我的大交通订单列表	需登录
GET	/api/order/{orderNo}/receipt	行程单 / 票据	需登录
GET	/api/passenger/list	常用旅客列表	需登录
POST	/api/passenger/add	新增常用旅客	需登录
DELETE	/api/passenger/{id}	删除常用旅客	需登录
GET	/api/price/trend	价格趋势数据	公开

酒店与目的地接口

方法	路径	描述	认证
GET	/api/hotel/search	酒店搜索	公开
GET	/api/hotel/{id}	酒店详情	公开
GET	/api/hotel/{id}/rooms	房型列表	公开
POST	/api/hotel/order/create	创建酒店订单（含 @RateLimiter）	需登录
POST	/api/hotel/order/{orderNo}/pay	模拟支付	需登录
POST	/api/hotel/order/{orderNo}/cancel	取消订单	需登录
GET	/api/hotel/orders	我的酒店订单列表	需登录
GET	/api/hotel/order/{orderNo}/receipt	酒店订单凭证	需登录
GET	/api/attraction/search	景点搜索	公开
GET	/api/attraction/{id}	景点详情	公开
POST	/api/attraction/{id}/ticket	景点门票购买入口	需登录
GET	/api/tour/list	一日游 / 周边游产品列表	公开
POST	/api/review/add	提交评价	需登录
GET	/api/review/list	评价列表	公开
POST	/api/review/report	举报评价	需登录

方法	路径	描述	认证
GET	/api/reply/list	商家回复列表	公开

AI 智能规划接口

方法	路径	描述	认证
POST	/api/ai/plan/generate	生成行程 (JSON Output)	需登录
GET	/api/ai/plan/list	我的行程列表	需登录
GET	/api/ai/plan/{id}	行程详情	需登录
POST	/api/ai/chat	AI 客服多轮对话	需登录
GET	/api/notification/list	通知列表	需登录
POST	/api/notification/read/{id}	标记通知已读	需登录
GET	/api/notification/unread-count	未读通知数	需登录

社区接口

方法	路径	描述	认证
GET	/api/post/list	帖子列表 (推荐流)	公开
GET	/api/post/{id}	帖子详情	公开
POST	/api/post/create	发布帖子 / 草稿	需登录
GET	/api/post/my	我的发布	需登录
GET	/api/post/following	关注流	需登录
POST	/api/like/toggle	点赞 / 收藏切换	需登录
GET	/api/like/status	点赞 / 收藏状态	需登录
GET	/api/like/my/collects	我的收藏	需登录
GET	/api/comment/list	评论列表	公开
POST	/api/comment/add	发表评论	需登录
DELETE	/api/comment/{id}	删除评论	需登录
POST	/api/follow/{userId}	关注/取关用户	需登录
GET	/api/follow/fans/{userId}	粉丝列表	公开
GET	/api/follow/following/{userId}	关注列表	公开

方法	路径	描述	认证
POST	/api/file/upload	上传图片	需登录

管理后台接口（需 role=1）

方法	路径	描述
GET	/api/admin/dashboard/data	可观测大屏数据
GET	/api/admin/stats	后台统计摘要
GET/POST/PUT/DELETE	/api/admin/flights/**	航班资源管理
GET/POST/PUT/DELETE	/api/admin/trains/**	火车资源管理
GET/POST/PUT/DELETE	/api/admin/hotels/**	酒店资源管理
GET/POST/PUT/DELETE	/api/admin/hotel-rooms/**	房型资源管理
GET	/api/admin/orders	订单流水查看
GET	/api/admin/posts	内容审核列表
POST	/api/admin/posts/{id}/approve	审核通过
POST	/api/admin/posts/{id}/reject	审核拒绝
GET	/api/admin/review-reports	评价举报工单
POST	/api/admin/review-reports/{id}/resolve	处理举报
GET	/api/admin/users	用户管理
POST	/api/admin/users/{id}/disable	禁用用户
POST	/api/admin/users/{id}/enable	启用用户
GET	/api/admin/logs	系统日志查看
GET/POST/DELETE	/api/admin/sensitive-words	敏感词管理
GET/POST/PUT/DELETE	/api/admin/coupons	优惠券管理

优惠券接口

方法	路径	描述	认证
GET	/api/coupon/list	优惠券列表	公开
GET	/api/coupon/my	我的优惠券	需登录
POST	/api/coupon/claim/{id}	领取优惠券	需登录

4.1.4 接口限流设计

@RateLimiter 注解用于高频写接口，RateLimiterInterceptor 基于 Redis 计数器 + IP 拦截超限请求：

```
// 创建订单接口：每个 IP 每秒最多 5 次
@RateLimiter(maxRequests = 5, timewindowSeconds = 1)
@PostMapping("/api/order/flight/create")
public Result<TrafficOrderVO> createOrder(@RequestBody TrafficOrderCreatedDTO dto) { ... }
```

超限时返回 HTTP 429，响应体 code=429，msg="请求过于频繁，请稍后再试"。

4.2 内部界面设计

4.2.1 服务间通信规范

- 服务间通信：采用 RESTful API，统一使用 JSON 数据格式交互。主外键命名统一采用小写下划线，时间字段统一约定为 create_time、update_time，确保各子系统联调时不出现数据割裂。
- 接口文档管理：由后端采用 Swagger/OpenAPI 自动生成接口文档，奉行"接口契约先行"策略，未完成的后端接口先使用 Apifox 等工具提供 Mock 数据支撑前端页面开发。
- 鉴权机制：通过 JWT 在 HTTP Header 中传递 Token，后端过滤器统一进行合法性与有效期校验。
- 异步处理机制（降级设计）：为降低验收环境搭建成本，不引入重量级消息队列（如 Kafka/RabbitMQ）。采用 Spring Boot 内部事件发布机制（ApplicationEvent）处理站内信通知、日志记录；采用 Redis 延时队列或 Spring 定时任务（@Scheduled）处理超时未支付订单的库存回滚。

4.2.2 数据库设计

本系统采用 MySQL 8.0，业务表主要使用 tm_ 前缀，系统日志与敏感词表沿用当前实现中的 sys_ 前缀。主键统一为 BIGINT AUTO_INCREMENT，公共字段按实际表结构包含 create_time、update_time（DATETIME）与逻辑删除字段 deleted（TINYINT DEFAULT 0）。

用户与权限域

表名	关键字段	说明
tm_user	id, username, password (BCrypt), email, phone, role(0 普通/1 管理员), avatar_url, status	账号主表
tm_follow	id, follower_id, following_id	关注关系，唯一约束 (follower_id, following_id)

大交通域

表名	关键字段	说明
tm_flight	id, flight_no, airline, departure_city, arrival_city, departure_time, arrival_time, economy_price, business_price, total_seats, available_seats, status	航班表

表名	关键字段	说明
tm_train	id, train_no, train_type, departure_station, arrival_station, departure_time, arrival_time, first_class_price, second_class_price, first_class_seats, second_class_seats, status	车次表
tm_traffic_order	id, order_no, user_id, order_type (0 机票/1 火车票), ticket_id, seat_type, passenger_name, passenger_id_card, amount, status, pay_time, create_time	大交通订单
tm_passenger	id, user_id, name, id_card, phone, type	常用旅客
tm_price_history	id, ticket_id, ticket_type, record_date, lowest_price	历史价格记录
tm_coupon	id, name, description, discount_type, discount_value, min_amount, expire_date, stock, status	优惠券
tm_user_coupon	id, user_id, coupon_id, status, received_time, used_time	用户领券记录

酒店与目的地域

表名	关键字段	说明
tm_hotel	id, name, city, address, star_rating, description, cover_img, lat, lng, avg_price, score, status	酒店主表
tm_hotel_room	id, hotel_id, room_type, bed_type, area, price, total_rooms, available_rooms, facilities, status	房型表，使用可售房量原子扣减防超卖
tm_hotel_order	id, order_no, user_id, hotel_id, room_id, hotel_name, room_type, check_in_date, check_out_date, nights, guest_name, guest_phone, amount, status, create_time	酒店订单
tm_attraction	id, name, city, address, description, cover_img, adult_price, child_price, total_tickets, available_tickets, open_time, lat, lng, status	景点表
tm_review	id, user_id, target_type, target_id, order_id, rating, content, images, tags, status	评价表，tags 为逗号分隔
tm_reply	id, review_id, user_id, content	商家回复

表名	关键字段	说明
tm_review_report	id, review_id, reporter_id, reason, status (0 待处理/1 已处理)	评价举报
tm_tour_product	id, name, description, tour_type, departure_city, destination, duration, price, cover_img, status	一日游/周边游产品

社区域

表名	关键字段	说明
tm_post	id, user_id, title, content, images, location, tags, like_count, comment_count, status (0 待审/1 已发/2 拒绝), view_count	游记帖子
tm_comment	id, post_id, user_id, parent_id (NULL 为顶级), content, like_count	树状评论
tm_like	id, user_id, target_type (POST/COMMENT/COLLECT), target_id	点赞 / 收藏记录

后台与监控域

表名	关键字段	说明
sys_log	id, method_name, class_name, params, result, duration_ms, user_id, ip, status, create_time	AOP 操作日志
sys_sensitive_word	id, word, level, create_time	敏感词库

关键索引

```
CREATE INDEX idx_flight_depart ON tm_flight(departure_city, arrival_city, departure_time);
CREATE INDEX idx_hotel_city ON tm_hotel(city);
CREATE INDEX idx_post_status ON tm_post(status, create_time);
CREATE INDEX idx_traffic_order_user ON tm_traffic_order(user_id, status);
CREATE INDEX idx_hotel_order_user ON tm_hotel_order(user_id, status);
CREATE INDEX idx_sys_log_time ON sys_log(create_time, status);
```

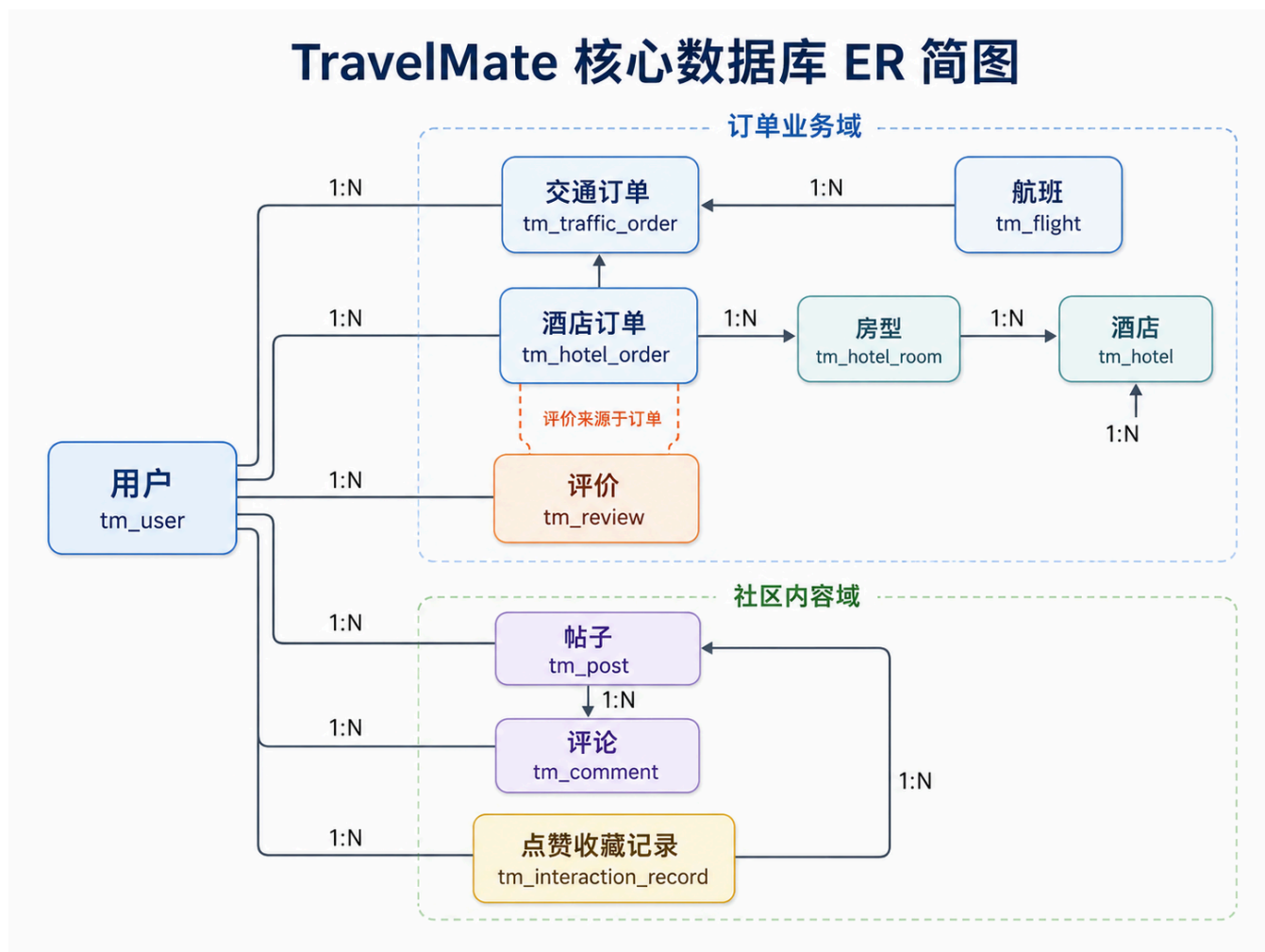
防超卖核心 SQL

数据库原子扣减库存（以酒店房型为例）：

```
UPDATE tm_hotel_room
SET available_rooms = available_rooms - 1
WHERE id = #{roomId}
AND status = 1
AND available_rooms > 0
```

Redis 原子预减 (Lua 脚本):

```
local current = redis.call('GET', KEYS[1])
if not current then
    return -2
end
current = tonumber(current)
if current <= 0 then
    return -1
end
redis.call('DECR', KEYS[1])
return current - 1
```



4.3 用户界面设计

- **风格与实现指南：**采用简洁卡片式设计，主色调为蓝色系，保证页面的可读性与交互效率。在前端开发中合理复用通用 UI 组件库（Element Plus）和图表可视化库（ECharts），不在 UI 原创上投入过多时间。
- **核心页面布局：**
 - **综合首页（A/B 模块）：**顶部多 Tab 搜索栏（机票/火车票/酒店）+ 核心功能入口区 + 底部推荐展位。
 - **交通搜索页（A 模块）：**`FlightSearch.vue` 与 `TrainSearch.vue` 分别提供航班 / 火车搜索、乘客选择、下单入口和价格趋势弹窗。

- **目的地页面 (B 模块):** `HotelSearch.vue` 提供酒店筛选; `HotelDetail.vue` 展示酒店详情、房型摘要、图文评价列表与评价提交入口; `AttractionList.vue` 展示景点与一日游 / 周边游产品。
- **AI 规划页 (C 模块):** `AiPlan.vue` 左侧为需求输入和多轮对话, 右侧展示生成的结构化行程卡片, 并关联站内酒店/航班等查询能力。
- **社区发现页 (D 模块):** `Community.vue`、`PostCreate.vue`、`PostDetail.vue` 支持双列瀑布流、图文发布、点赞收藏、关注与树状评论。
- **用户订单页 (A/D 模块公共):** `MyOrders.vue` 顶部状态筛选标签 (待支付/已支付/已出行/退款) + 订单列表卡片 + 核心操作按钮 (去支付、取消、查看凭证)。
- **后台管理 (E 模块):** `AdminDashboard.vue` 采用左侧折叠菜单 + 右侧内容区布局, 包含可观测仪表盘、航班/火车/酒店/房型 CRUD、优惠券、订单流水、帖子审核、评价举报工单、敏感词和系统日志。

5 异常处理设计

5.1 出错信息管理

- **统一响应格式与状态码规范:**
前后端统一定义全局响应格式结构:

```
{
  "code": 200,
  "msg": "错误描述/成功提示",
  "data": null
}
```

- **核心异常分类枚举:**
 - 401: 认证失败/Token 过期 (跳转登录页)
 - 403: RBAC 权限不足 (拒绝访问特定接口)
 - 404: 请求的 Mock 资源或路径不存在
 - 429: 接口限流拦截 (超过 RateLimiter 阈值)
 - 500: 服务器内部兜底错误
 - 601: 库存不足/超卖拦截 (针对小并发订单防超卖设计)
 - 自定义错误码说明: `ERR_001` AI 服务暂不可用; `ERR_002` 库存不足, 订单关闭; `ERR_003` 登录过期, 请重新认证。
- **异常处理机制:**
 - **后端:** 通过 `@RestControllerAdvice` 结合统一异常处理类 (`GlobalExceptionHandler`) 集中捕获自定义业务异常和运行时异常, 并转化为规范的 JSON 响应, 不将数据库原生报错栈暴露给前端。特殊处理: 数据库连接失败提示检查 `DB_PASSWORD`; 表不存在提示执行 `docs/sql/init.sql`。
 - **前端:** 通过 Axios 全局拦截器 (Interceptors) 统一拦截响应, 根据 `code` 码触发统一的 UI Toast 错误提示或路由重定向。

5.2 故障预防与补救

- **AI 服务降级与容错**：当 DeepSeek API 接口超时、触发并发限流或 JSON 结构化输出失败时，系统**自动降级切换为本地预设的静态推荐模板或默认行程方案**，并辅以"人工校验提示"，确保 AI 规划核心页面在演示阶段的可用性。调用时需严格控制 `max_tokens` 以避免输出被中途截断导致 JSON 损坏。
- **库存防并发与回滚**：
 - 在订票、订房场景中，后端采用**数据库原子扣减（UPDATE...WHERE available > 0）**预防超卖故障；酒店房型额外通过 Redis Lua 进行预减，数据库侧最终校验，形成双重保障。
 - 利用 Spring `@Scheduled` 定时任务扫描待支付订单，确保订单超过 15 分钟未完成模拟支付时，**自动触发库存回滚机制**释放预占资源。
- **一致性问题排查**：强制要求主外键关系（如订单表与模拟资源表）在插入时进行程序逻辑校验，防止孤儿数据导致联调崩溃。
- **数据保障（降级）**：由项目统一负责人提供包含**完整数据结构与初始测试数据（Mock 数据）的 SQL 初始化脚本**（`docs/sql/init.sql`），保障项目在不同成员本地以及助教验收环境中的可重现启动。

5.3 系统维护设计

- **轻量日志审计**：不使用 ELK 栈，改为采用 Spring Boot 本地日志文件记录（Logback），并结合 **Spring AOP 切面（SysLogAspect）** 实现系统关键操作的日志审计，将关键记录持久化到数据库 `sys_log` 表中。
`SysLogAspect` 环绕通知自动拦截所有 `com.travelmate.controller` 下的方法，记录方法名、参数摘要、耗时、操作用户、IP 及成功/失败状态，日志落库失败不影响业务。
- **轻量监控大屏（E 模块职责）**：在管理后台提供自研的数据可观测仪表盘。不引入第三方 APM 探针，基于 Redis 和本地应用内存，轻量级统计并展示：系统 QPS、热门查询词频、实时报错日志列表（便于助教排查验收）以及订单财务图表，前端使用 ECharts 可视化。
- **接口限流控制**：通过 `@RateLimiter` 自定义注解结合 Redis 计数器，对下单等敏感接口配置 QPS 限流防刷，超限返回 HTTP 429。
- **数据库维护**：提供 `setup.ps1` 脚本支持一键初始化/重建数据库，使用 `mysql.exe` 直接 `SOURCE` 导入以保证中文数据编码正确。

6 性能优化和安全设计

6.1 性能优化

- **Redis 缓存热点数据**：对于高频访问且极少变动的基础 Mock 数据（如热门目的地、首页推荐聚合、基础价格查询）以及系统全局配置，使用 Redis 进行缓存。用户会话状态管理采用无状态 JWT 结合 Redis 黑名单实现。
- **前端资源优化**：由于不部署真实 CDN 节点，前端性能优化通过 Vue 路由的**页面组件懒加载**、Vite 构建时的代码分包（Code Splitting）以及在网关层/Nginx 配置 HTTP Gzip 压缩来实现。
- **图片与 UGC 内容压缩**：针对旅途社区的 UGC 图文发布，前端在上传前进行图片尺寸裁切与压缩，减少服务器带宽与存储占用。
- **防并发与小规模削峰**：在订房场景，通过 Redis Lua 进行**预占库存的快速校验**，阻挡无效的高并发写请求直达数据库；机票/火车票和酒店最终均通过数据库原子 UPDATE 完成库存校验；通过 Spring 定时任务处理超时取消和库存回滚。

6.2 安全与风险控制设计

- **数据安全**：用户密码须采用加盐哈希算法（BCrypt）进行加密存储，不明文保存敏感信息。数据库密码、API Key 等敏感配置通过 `.env` 文件注入环境变量，不写入代码仓库。
- **认证与会话安全**：采用 JWT（jsonwebtoken 0.11.5，HS256）生成无状态 Token，过期时间 24h。每次重启生成新密钥，旧 token 全部失效（课程场景可接受）。
- **权限安全控制（RBAC）**：`SecurityConfig` 按接口路径配置白名单，其余接口均需认证；管理员接口额外校验 `role == 1`。确保普通用户无法越权操作管理后台的审核与资源维护接口。
- **社区内容安全审核**：采用"敏感词词库自动过滤拦截（SensitiveWordService）+ 后台人工审核 workflow"双重机制。管理员可动态维护敏感词库，立即生效。
- **防超卖安全**：利用数据库原子 UPDATE 条件（如 `available_rooms > 0`、`available_seats > 0`）保障订单并发创建时的数据一致性；酒店房型 Redis 侧先行预减作为第一道防线。
- **防刷与限流**：前端实施按钮防抖拦截，后端 `@RateLimiter` 注解 + Redis IP 频次限制（单 IP 每秒最多 5 次下单请求），超限返回 HTTP 429。

7 项目测试计划

序号	测试类型	对应子系统	测试内容	验收方法	计划时间
1	单元测试	认证/安全配置	BCrypt 密码校验、JWT 生成与解析、SecurityConfig 白名单与管理权限	JUnit 5 单元用例	第 9 周
2	接口测试	核心 Controller	登录注册、航班/火车搜索、酒店搜索、评价提交、后台审核接口响应格式	Apifox/Postman 脚本	第 10 周
3	集成测试	前后端联调	酒店搜索 → 下单 → 支付 / 取消、航班火车下单、社区发帖评论	全流程手工演练	第 10 周
4	并发/性能测试	订单库存模块	50 线程并发下单，验证航班/火车票和酒店房型无超卖	JMeter 压测报告	第 11 周
5	安全测试	认证/权限模块	Token 篡改、未登录访问写接口、普通用户越权访问 <code>/api/admin/**</code>	手工渗透 + 代码审查	第 11 周
6	系统测试	全系统	登录 → AI 规划 → 搜索 → 下单 → 支付 → 评价 → 后台审核 / 举报处理	端到端演示录屏	第 12 周
7	降级验证	AI 模块	未配置或断开 DeepSeek API 时系统自动返回模板行程，主流程不报错	手工验证	第 12 周

项目测试计划闭环流程图



验收前补充改进项：

- **计划书与文档一致性：**需求规格说明、详细设计说明、README、测试报告和实施计划需统一接口路径、数据库表名、模块分工和技术栈版本，避免同时出现早期规划路径与当前实现路径。
- **演示脚本补充：**将“登录 → AI 规划 → 搜索 → 下单 → 支付 / 取消 → 评价 → 后台审核 / 举报处理”整理为固定演示流程，明确每一步使用的账号、页面、接口和预期结果。
- **测试证据补充：**测试报告中应补充后端测试、前端构建、数据库重建、中文数据校验、50 并发无超卖、AI 降级和管理员端到端回归记录。
- **接口契约沉淀：**当前接口以代码和前端调用为准，后续可补充 Apifox/OpenAPI 文档，减少详细设计与实现再次偏离。
- **AI 使用附件：**单独整理 Prompt 模板、JSON Output 约束、Function Calling 工具定义、降级模板和 API Key 配置说明，增强大模型使用的可解释性。
- **项目实现优化：**后续可拆分体量较大的 `AdminController`，补独立订单详情页，增强 AI 结果与预订资源的联动，并继续优化移动端适配、空状态、错误提示和图片上传失败重试等体验细节。